

MEQI

Magnetic Field Controller

GMW 3470 electromagnet · Kepco BOP supply · NI USB-6259 DAQ

USER & OPERATOR MANUAL



TRMOKE transition project · Python control software. The screenshots show the built-in simulator; the real instruments behave identically.

Contents

1. What this is
2. Installing & first run
3. Running the software
4. The Control tab
5. Setting a field & how the seek works
6. Demagnetising
7. Calibration
8. The AUX I/O tab
9. Settings & appearance (light / dark)
10. Remote operation over the network
11. The communication protocol (for scripting)
12. Safety behaviour
13. Troubleshooting
14. Quick reference

1. What this is

MEQI drives an electromagnet to a requested magnetic field and holds it there.

A **Kepco BOP** power supply (over GPIB) pushes current through a **GMW 3470** dipole electromagnet, and a Hall probe read through an **NI USB-6259** DAQ card measures the resulting field. You ask for a field in millitesla; the software works out the current, ramps to it, and fine-tunes with a PID loop until the field is stable.

The software is organised as **one service and any number of clients**. The service is the single program that actually owns the hardware. Clients — the graphical window, a text console, or your own scripts — connect to it and send commands. That is what lets you watch and drive the magnet from the lab PC or from another machine on the network at the same time.

Key principle — set and report. The magnet is "fire and forget": when you set a field it accepts the command immediately and starts moving. It does *not* wait until the field is stable. You (or an automation script) check the reported status to see when it has settled. The magnet never decides what happens next — it only sets and reports.

2. Installing & first run

The project uses **uv**, a fast Python environment manager. Install it once (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Close and reopen PowerShell so it sees **uv**. Then move into the project folder and install the graphical extras (a one-time ~100 MB download of the GUI libraries):

```
cd "C:\Users\<you>\...\meqi-control"  
uv sync --extra gui
```

uv sync builds a private environment for the project and installs everything into it. You never touch your system Python.

OneDrive note. If the project lives in a OneDrive folder you may see **Access is denied (os error 5)** when **uv** rebuilds — OneDrive locks the environment files while syncing. Fix it once by keeping the environment in fast local storage:

```
[Environment]::SetEnvironmentVariable('UV_PROJECT_ENVIRONMENT', "$env:LOCALAPPDATA\uv-venvs\meqi-control", 'User')  
$env:UV_PROJECT_ENVIRONMENT = "$env:LOCALAPPDATA\uv-venvs\meqi-control"  
uv sync --extra gui
```

Or launch everything through the included **dev.ps1** helper, which does this automatically: **.\dev.ps1 run scripts/run_gui.py**.

3. Running the software

There are two ways to run: **self-contained** (the window drives its own built-in magnet) and **service + clients** (one program owns the magnet, others connect to it).

Just the window (simplest)

```
uv run scripts/run_gui.py
```

This opens the control window running against the built-in simulator — no service, no hardware needed. Perfect for learning the interface.

Service + clients

Start the service in one PowerShell window; it owns the magnet:

```
uv run scripts/run_service.py
```

Then connect any number of clients from the same PC (or another machine) with `--connect`:

```
uv run scripts/run_gui.py --connect localhost      # the window
uv run scripts/magnet_console.py --connect localhost # a text console
```

Why nothing showed up. Running `run_gui.py` with *no* arguments starts its *own* separate simulated magnet. If a service is running and you want the window to drive *that* magnet, add `--connect`. One service = one magnet = the single source of truth; the window and the console are just two views onto it.

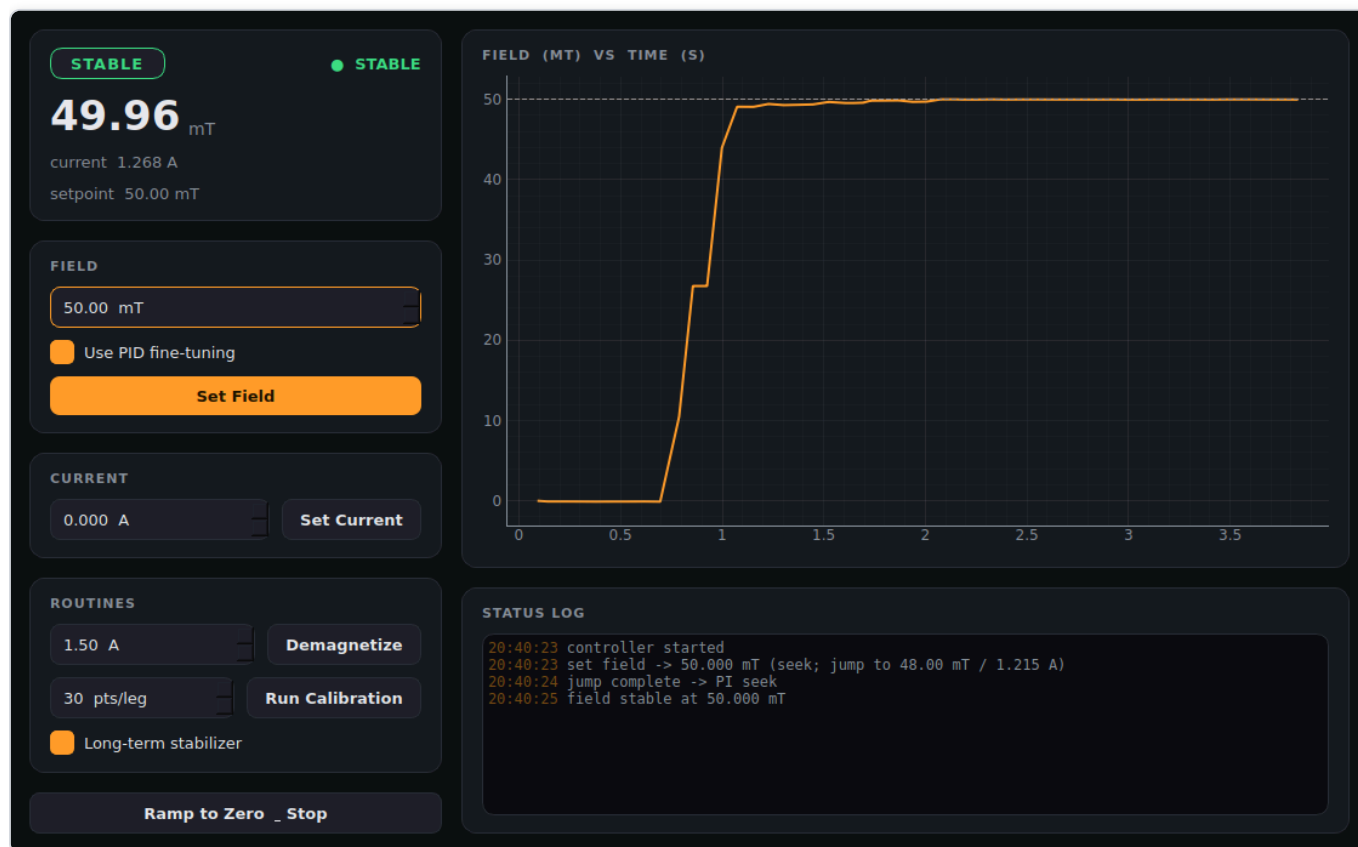
Choosing a theme at startup

```
uv run scripts/run_gui.py --theme light
uv run scripts/run_gui.py --theme dark
```

The theme can also be set in Settings and remembered between launches (Section 9).

4. The Control tab

The main window opens on the **Control** tab. Everything you need for day-to-day field control is here; live values update several times a second.



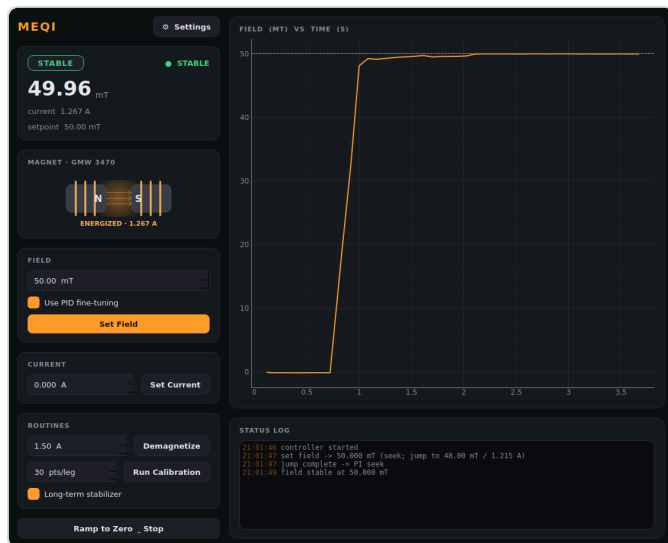
The Control tab, holding +50 mT. The plot shows the field reaching the setpoint; the left column has the readouts and every control.

Readouts (top-left)

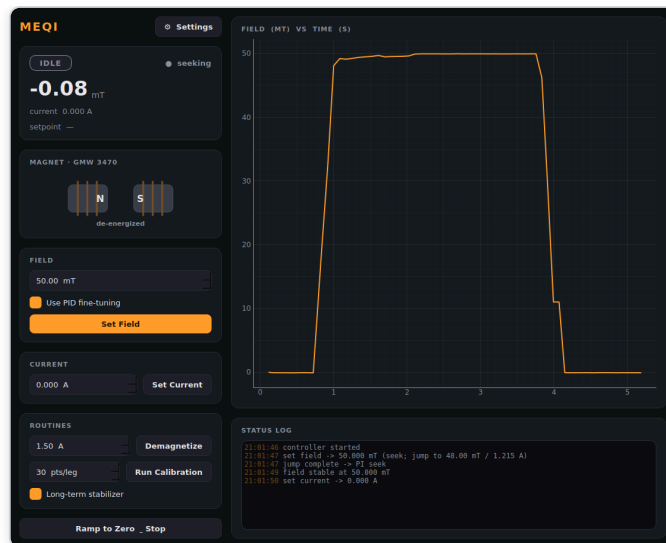
The big number is the measured field. Below it are the coil current and the active setpoint. The **state badge** and the **STABLE lamp** tell you what the controller is doing: it turns green and reads STABLE once the field has held within tolerance long enough.

The magnet indicator

The **GMW 3470** panel is a live picture of the magnet. Its gap glows amber and reads ENERGIZED whenever current is flowing; the N/S labels follow the polarity and the brightness tracks the current. At zero current it dims to de-energized.



Energised — current flowing, poles N–S.



De-energised — ramped to zero.

Controls (left column)

- **Field** — type a value in mT and press **Set Field**. Leave **Use PID fine-tuning** ticked for the tightest result (Section 5).
- **Current** — set the coil current directly in amps, bypassing the field logic.
- **Demagnetize** — run the degauss routine at the given amplitude (Section 6).
- **Run Calibration / View curve...** — measure or inspect the B(I) curve (Section 7).
- **Long-term stabilizer** — a slow watchdog that trims out drift while holding.
- **Ramp to Zero & Stop** — safely returns the coil to zero.

Status log (bottom)

Every command and event is timestamped here. Faults — such as a value over the current limit — are printed in red.

5. Setting a field & how the seek works

When you set a field with PID fine-tuning on, the controller runs a deliberate three-step sequence. Understanding it explains what you see on the plot.

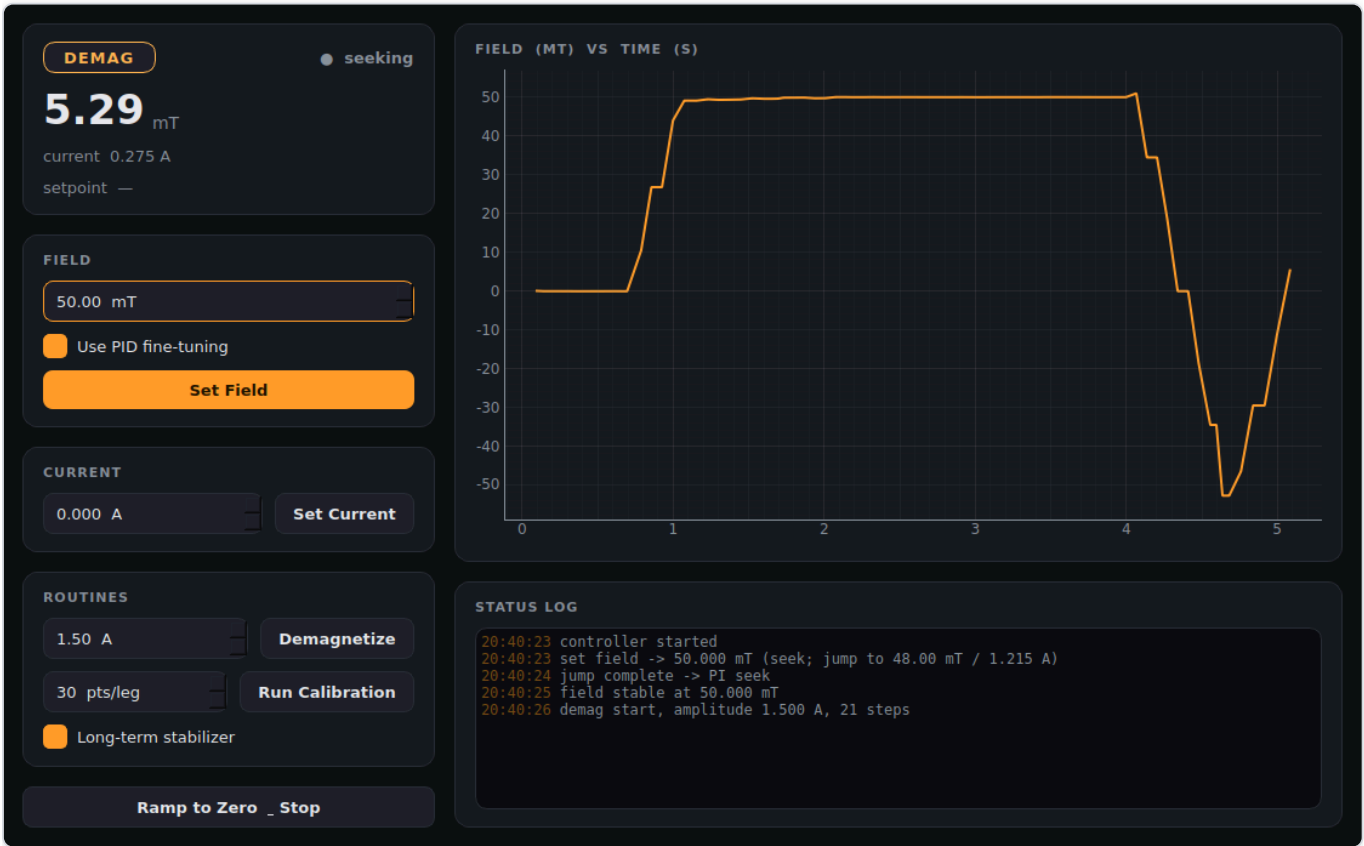
1. **Calibration jump.** Using the measured $B(I)$ curve it jumps the current to land just *short* of the target — deliberately undershooting by a small "field step" so it always approaches from one side.
2. **PID seek.** A PI controller then closes the last couple of mT, moving in one direction only so it never overshoots.
3. **Freeze & confirm.** Once inside tolerance it *freezes* the current and switches to a slower, cleaner measurement to confirm. If the field holds within tolerance for the dwell time, the **STABLE** lamp lights.

Why it freezes. The magnet's iron has hysteresis. If the controller kept nudging the current up and down to chase measurement noise, the field would wobble by more than the tolerance. Freezing the current once close is what lets it hold a field tighter than the noise — the same reason the original LabVIEW system "stopped the PID" near target.

The state badge walks through IDLE → RAMPING → SEEK → STABLE. A typical seek settles in one to two seconds. If the field later drifts, the fast PID does *not* re-engage; only the gentle long-term stabilizer trims it, so the field is never kicked around unexpectedly.

6. Demagnetising

Press **Demagnetize** with an amplitude in amps. The controller drives the coil through **decaying, alternating current steps** that shrink toward zero, leaving the magnet in a clean, low-remanence state. It ramps between steps and waits for each current to arrive.



A demag in progress — the field oscillates with shrinking amplitude down toward zero. The state badge reads DEMAG; when it finishes the coil is at 0 A.

7. Calibration

The field controller needs a measured **B(I) curve** — how much field each current produces. Press **Run Calibration** to sweep the magnet through a full cycle and build it. Duplicate currents from the up and down legs are averaged to cancel hysteresis; you can subtract remanence so zero current maps to zero field.

When the sweep finishes the curve opens automatically. You can also open it any time with **View curve...**



The calibration viewer: measured points, the interpolated curve, and the Hall-probe parameters the curve was taken with.

Saving & loading

In **Settings** → **Calibration** you can **Save...** a curve to a text file and **Load...** one back. The file also stores the Hall-probe parameters it was recorded with — on load you are asked whether to adopt those too, or keep your current ones and just view the curve as recorded.

Hall parameters. The Hall-probe sensitivity, geometry correction and zero offset convert the probe voltage into field. If you change them (Settings → Hall probe), re-run the calibration afterwards so the B(I) curve matches.

8. The AUX I/O tab

The same DAQ card has spare BNC connectors. The **AUX I/O** tab exposes them, all ± 10 V and all readable and controllable — locally or over the network.

The screenshot displays the AUX I/O tab interface, which is divided into three main sections: Analog Outputs, Analog Inputs, and Digital Outputs.

ANALOG OUTPUTS (± 10 V)

Channel	Value	Action	Commanded Value
ao0	0.000 V	Set	cmd +2.500 V
ao1	0.000 V	Set	cmd -4.250 V
ao2	0.000 V	Set	cmd +0.000 V
ao3	0.000 V	Set	cmd +9.999 V

ANALOG INPUTS (± 10 V, RSE)

Channel	Value
ai1	+1.864 V
ai2	+1.106 V
ai3	+1.256 V

DIGITAL OUTPUTS

Channel	State
line0	On
line1	Off
line2	On

A **Read now** button is located below the analog inputs section.

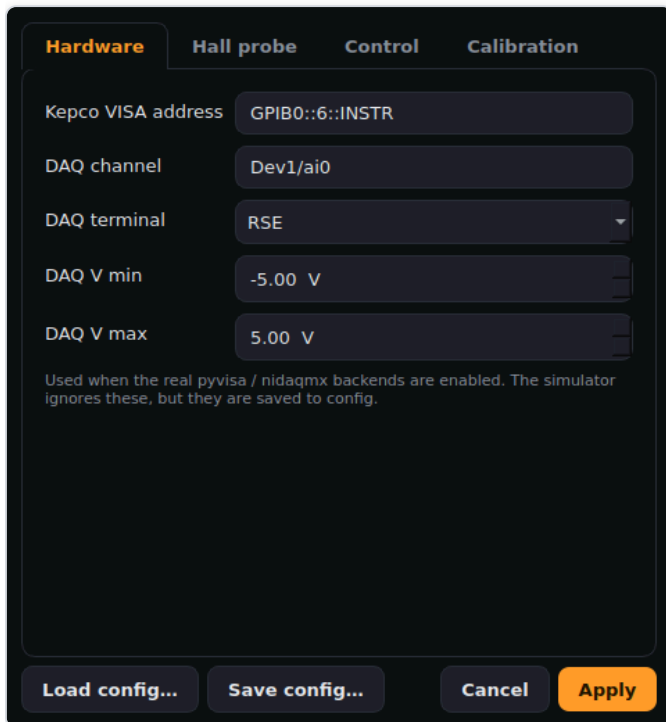
The AUX tab: four analog outputs with commanded read-back, three live analog inputs, and three digital-output toggles.

- **Analog outputs** (ao0–ao3) — type a voltage and press **Set**; the commanded value is shown to the right.
- **Analog inputs** (ai1–ai3; ai0 is reserved for the Hall probe) — live readings, plus **Read now** for an explicit single sample.
- **Digital outputs** — three toggles reflecting and setting line state.

Which channels appear is set in the configuration, so you can match your own wiring without editing code.

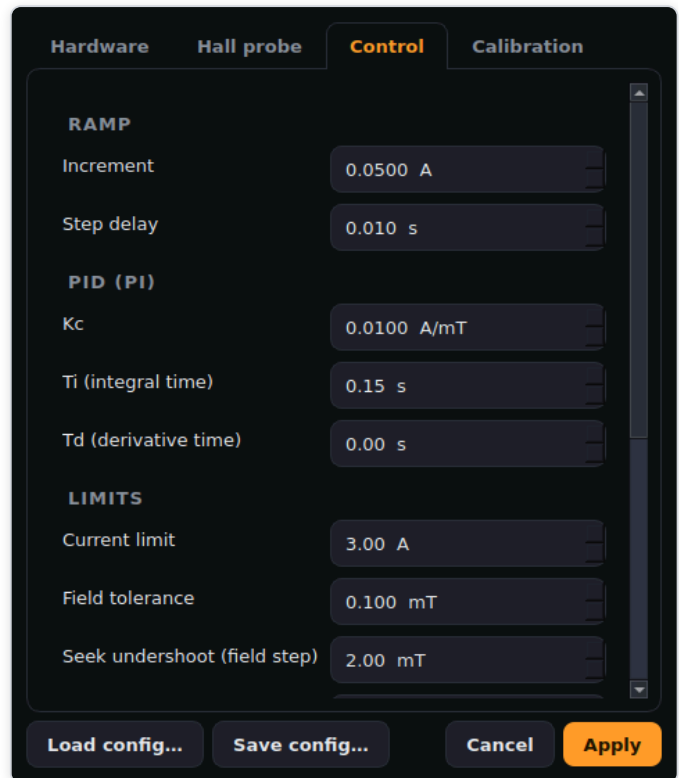
9. Settings & appearance

The gear button opens **Settings**, organised into tabs.



The screenshot shows the 'Hardware' tab of the Settings window. It has four sub-tabs: 'Hardware' (selected), 'Hall probe', 'Control', and 'Calibration'. The 'Hardware' tab contains five input fields: 'Kepco VISA address' (GPIB0::6::INSTR), 'DAQ channel' (Dev1/ai0), 'DAQ terminal' (RSE), 'DAQ V min' (-5.00 V), and 'DAQ V max' (5.00 V). Below these fields is a note: 'Used when the real pyvisa / nidaqmx backends are enabled. The simulator ignores these, but they are saved to config.' At the bottom are four buttons: 'Load config...', 'Save config...', 'Cancel', and 'Apply'.

Hardware — Kepco address, DAQ channel/range.



The screenshot shows the 'Control' tab of the Settings window. It has four sub-tabs: 'Hardware', 'Hall probe', 'Control' (selected), and 'Calibration'. The 'Control' tab is divided into three sections: 'RAMP', 'PID (PI)', and 'LIMITS'. The 'RAMP' section has 'Increment' (0.0500 A) and 'Step delay' (0.010 s). The 'PID (PI)' section has 'Kc' (0.0100 A/mT), 'Ti (integral time)' (0.15 s), and 'Td (derivative time)' (0.00 s). The 'LIMITS' section has 'Current limit' (3.00 A), 'Field tolerance' (0.100 mT), and 'Seek undershoot (field step)' (2.00 mT). At the bottom are four buttons: 'Load config...', 'Save config...', 'Cancel', and 'Apply'.

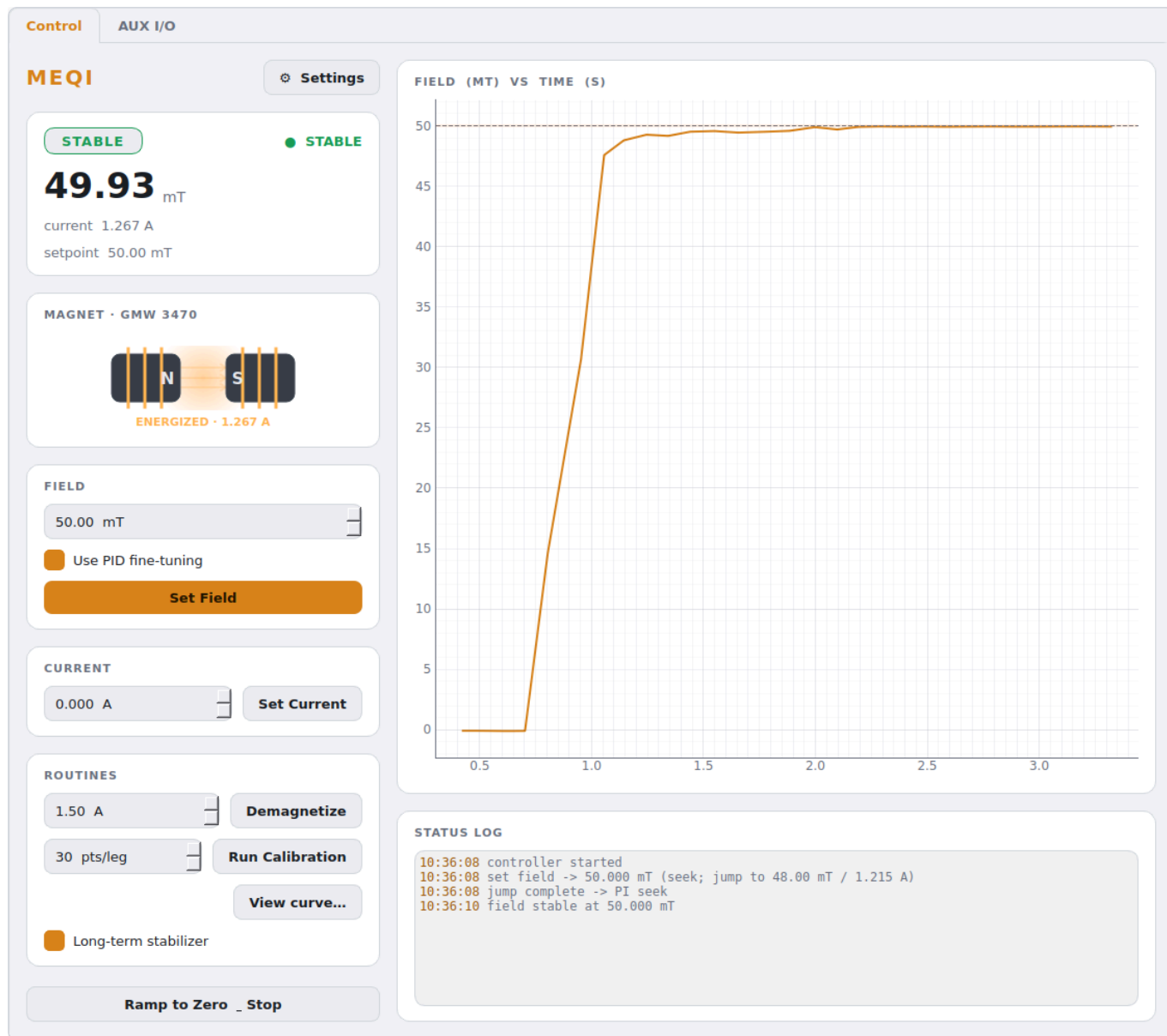
Control — ramp, PID, limits, acquisition.

- **Hardware** — Kepco VISA address, DAQ channel, terminal and voltage range (used with real hardware).
- **Hall probe** — the volts-to-field conversion (this calibrates the field *measurement*).
- **Control** — ramp increment/delay, PID gains, current limit and tolerances, acquisition profiles.
- **Calibration** — save / load / view the B(I) curve.
- **Appearance** — choose a light or dark theme.

Load config... / **Save config...** at the bottom write all of the above to a plain-text file.

Light or dark

Pick the theme on the **Appearance** tab and **Save config** to remember it; it takes effect on the next launch. Or override once at startup with `--theme light`.



The same Control tab in the light theme.

10. Remote operation over the network

Because the service binds to all network interfaces, the exact same setup works on the same PC or across the lab network — only the address the client dials changes.

```
# on the PC wired to the magnet:  
uv run scripts/run_service.py  
  
# from another machine on the network:  
uv run scripts/run_gui.py --connect 192.168.1.42
```

A connected window is a full client: the plot, all controls, Settings, the calibration curve and the AUX tab all work over the socket.

The screenshot shows the 'Control' tab of the MEQI Magnet Field Controller GUI. The interface has a dark theme with orange accents. At the top, there are four tabs: 'Hardware', 'Hall probe', 'Control' (which is selected and highlighted in orange), and 'Calibration'. Below the tabs, the settings are organized into three sections: 'RAMP', 'PID (PI)', and 'LIMITS'. Each section contains several parameters with corresponding input fields. A vertical scrollbar is on the right side of the settings area. At the bottom, there are four buttons: 'Load config...', 'Save config...', 'Cancel', and 'Apply' (which is highlighted in orange).

Section	Parameter	Value
RAMP	Increment	0.0500 A
	Step delay	0.010 s
PID (PI)	Kc	0.0100 A/mT
	Ti (integral time)	0.15 s
	Td (derivative time)	0.00 s
LIMITS	Current limit	3.00 A
	Field tolerance	0.100 mT
	Seek undershoot (field step)	2.00 mT

Settings edited on a remote window — changes are sent to the service, which applies them to the live controller.

Shared control. Any connected window can change settings on the live magnet. With two operators connected at once, the last change wins. Fine for a single operator; worth remembering if you share the service.

11. The communication protocol

Clients talk to the service over two ZeroMQ sockets: a **command** socket (request/reply, JSON) on port **5555**, and a **status** broadcast (publish/subscribe) on port **5556**. The protocol is plain JSON, so anything that speaks ZeroMQ — any language — can drive the magnet.

Commands (port 5555, JSON in → JSON reply)

Every reply is an object with an **ok** flag. A command returns `{"ok": true}` as soon as it is *accepted* — not when the motion finishes.

cmd	fields	does
<code>set_field</code>	field_mT, use_pid	seek to a field
<code>set_current</code>	current_A	set coil current directly
<code>demag</code>	amplitude_A	run the degauss routine
<code>calibrate</code>	n_per_leg, dwell_s	measure a new B(I) curve
<code>set_stabilizer</code>	enabled	long-term stabilizer on/off
<code>set_lock</code>	locked	external-control lock flag
<code>aux_set_ao</code>	channel, volts	set an analog output
<code>aux_set_do</code>	line, state	set a digital output
<code>aux_read_ai</code>	channel	read one analog input
<code>status</code> / <code>info</code>	—	one snapshot / field range & limits
<code>get_config</code> / <code>set_config</code>	(config)	read / write all settings
<code>get_calibration</code> / <code>set_calibration</code>	(curve)	read / write the curve

Status (port 5556, broadcast at 10 Hz)

Topic `status` carries: `state`, `setpoint_field_mT`, `measured_field_mT`, `current_A`, `field_stable`, `locked` and `aux` (the AO/AI/DO values). Topic `event` carries log lines.

Try it by hand — the console

```
uv run scripts/magnet_console.py --connect localhost
magnet> field 50          # set 50 mT (accepted immediately)
magnet> watch 5          # stream live status + events for 5 s
magnet> status            # one snapshot: is it stable yet?
magnet> ao 0 2.5         # analog out ao0 -> 2.5 V
magnet> do 0 on          # digital line0 high
magnet> current 0        # ramp coil to zero
```

From your own Python (no project code needed)

```
import zmq, json
s = zmq.Context().socket(zmq.REQ)
s.connect("tcp://localhost:5555")
s.send_json({"cmd": "set_field", "field_mT": 50.0, "use_pid": True})
print(s.recv_json())      # -> {'ok': True}
```

Automation pattern. To sequence the magnet with a measurement: send `set_field` , then read the status broadcast until `field_stable` is true, then trigger your measurement. The magnet stays "dumb"; the sequencing lives in your script.

12. Safety behaviour

- **Current limit.** Requests beyond the configured limit (default 3 A) are clamped silently and the event is logged in red.
- **Safe shutdown.** Closing the window, or the service stopping, **ramps the coil to zero and turns the output off** — current is never dropped abruptly.
- **Lost client / crash.** The service brings the magnet to a safe state on shutdown or a fault, so a dropped connection does not leave the coil energised unattended.
- **Ramping only.** The current always moves in small steps, never in a single jump.

13. Troubleshooting

Symptom	Cause & fix
Access is denied (os error 5) on <code>uv run</code>	OneDrive (or a network drive) is locking the environment. Keep the environment in local AppData (Section 2) or launch via <code>dev.ps1</code> .
The window doesn't reflect commands from the console	The window is running its own local simulator. Restart it with <code>--connect localhost</code> so it drives the same magnet as the service.
no reply (is the service running?)	The service isn't running, or the host/ports are wrong. Start <code>run_service.py</code> and check the <code>--connect</code> address and ports (5555 / 5556).
Address already in use starting the service	An earlier service still holds those ports. Close it, or use different <code>--cmd-port</code> / <code>--pub-port</code> .
GUI won't start — PySide6 missing	Run <code>uv sync --extra gui</code> once to install the GUI libraries.

14. Quick reference

Commands

```
uv sync --extra gui                # one-time install
uv run scripts/run_gui.py          # window, own simulator
uv run scripts/run_gui.py --connect HOST # window, drive a remote service
uv run scripts/run_gui.py --theme light # light theme this launch
uv run scripts/run_service.py       # the magnet service
uv run scripts/magnet_console.py --connect HOST # text console
uv run pytest                      # run the test suite
```

Network

Command socket (REQ/REP): **tcp://<host>:5555** · Status broadcast (PUB/SUB, 10 Hz): **tcp://<host>:5556**. The service binds **0.0.0.0**, so localhost and the lab network use identical code.

Defaults

Current limit 3 A · field tolerance 0.1 mT · seek undershoot 2 mT · ramp step 0.05 A every 10 ms · AUX ao0–ao3, ai1–ai3, 3 digital lines, all ± 10 V RSE.

MEQI magnet controller · part of the TRMOKE transition Python instrument suite. This manual documents the simulated-hardware build; the real Kepco / NI backends are enabled when the hardware is connected.